D2876R0
Proposal to extend `std::simd` with more constructors and accessors

Published Proposal, 2023-05-18

This version:
https://isocpp.org/files/papers/P2876R0.html

Authors:
Daniel Towner (Intel)
Matthias Kretz (GSI)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Proposal to provide additional constructors and accessors for `std::simd`

Table of Contents

§ 1. Motivation

The [P1928R3] proposal outlines a number of constructors and accessors which can be used to move data in and out of a `simd` object. However, there are a number of other data types in the standard C++ libraries which provide a form of data-parallel value storage, such as `std::bitset` and types which implement the `std::contiguous_range` concept such as `std::span`. It is desirable to be able to easily convert those to and from `std::simd` values too. In this paper we shall examine the benefits of providing constructors for building `simd` or `simd_mask` values from other types, and for adding accessors which allow them to be converted back into other types from their equivalent SIMD values. Some of these proposals were briefly outlined in [P2638R0], and some proposals are new.

§ 2. Bitset

`std::bitset` has many characteristics in common with a `simd_mask` value and it is useful to be able to use a `std::bitset` value as a mask and vice versa. There is no easy way for a programmer to concisely and efficiently achieve these interchange conversions so we propose that a constructor and an accessor are provided in `std::simd`. Firstly, a constructor can be provided for a `simd_mask`:

```
constexpr simd_mask(const std::bitset<size>& b);
```

This constructor allows a bitset with the same number of elements as a SIMD value to be used to build a `simd_mask` for that value. Each element in the constructor mask has the same boolean value as the respective element from the incoming bitset. It is not marked as explicit, so a bitset can be conveniently used anywhere that a simd_mask could be.

A `simd_mask` can be converted into an equivalent `std::bitset` using a conversion operator:

```
constexpr simd_mask::operator std::bitset<size>() const;
```

or alternatively though a named method which makes it explicitly clear that the conversion is happening:

```
constexpr std::bitset<size> simd_mask::to_bitset() const;
```

The output bitset value will have the same size as the simd_mask, and every element in the bitset will have the same value as its respective simd_mask element.

§ 2.1. Implementation experience

When working with ABIs which already have compact bit representations (e.g., AVX512 predicate registers) then converting to and from bitset is efficient. Conversion to and from wide element representations of masks (e.g., SSE or AVX), is more expensive but the `std::simd` library implementation is able to exploit the internal implementation details to make it more efficient than anything that the user could do using the public `std::simd` API.

§ 3. Conversion to and from integral value bit representations

There are several ways that `simd_mask`-like values could be stored, ranging from wide-element values (e.g., SSE, AVX), compact mask (e.g., ARM or AVX-512 predicates), bitsets, or byte-valued memory regions (e.g., `std::simd_mask::copy_to`). The `std::simd` API already has the ability to convert wide elements to and from simd_masks using, for example, the `operator+`. However, it can be useful to be able to convert to and from compact masks represented using raw bits stored in an integral value (something which `std::bitset` also supports explicitly). In this section we propose ways of constructing and accessing packed bits stored in integral values.

§ 3.1. Building a mask from a compact bit representation

When working with SIMD values of fixed sizes (rather than native types whose size can vary by target) it can be useful to express a mask pattern directly. For example, suppose a programmer requires a custom bit pattern, such as `0b10101101011010010101`. There is currently no easy or direct way to encode that pattern into a simd_mask value. For example, `simd_mask` has constructors which take a generator or a byte-valued memory region as input, but neither of these lend themselves to concisely encoding an arbitrary bit pattern.

We propose that the following constructor could be provided:

```
constexpr simd_mask(auto std::unsigned_integral bits) noexcept;
```

The simd_mask is constructed such that the first (rightmost, least significant) M bit positions are set to the corresponding bit values of `bits`, where M is the smaller of N and the number of bits in the value representation of `pattern`. If M is less than the size of the mask then the remaining bit positions are initialized to zeroes.

The issue with using a `std::unsigned_integral` as the container for the input is that it might contain too many bits for a small mask, or too few bits for a big mask. Without a way of representing an arbitrary number of bits in an integral value (e.g., C23's bit-precise `_BitInt`) we are limited to only allowing up to 64-bits to be inserted into a simd_mask, analagous to the same limits as a `std:bitset`.

§ 3.2. Compact bit accessor

There is currently no easy way for mask bits to be extracted from a `simd_mask` in a compact form. Neither of the existing methods to extract is efficient when used for this. For example, a mask's values

can becauseextracted a an array of bytes:

```
auto simd_mask<float> mask = ...;

bool maskAsBytes[1024];
mask.copy_to(maskAsBytes);
```

This copies each mask element to a byte in the supplied output iterator, but there is no easy way to go from that form to a compact bit representation.

The other way to extract the contents of a simd_mask is to convert it into a SIMD value representation. In the following example the mask contents are converted to elements which are either 0.0f or 1.0f according to their respective mask bit:

```
auto simd_mask<float> mask = ...;

simd<float> output = +mask;
```

But this doesn't allow easy conversion to a copmact set of bits either.

To make it possible to extract compacted bits as an unsigned integral value we propose to borrow an idea from `std::bitst`and provide:

```
constexpr std::uint64_t simd_mask::to_ullong() const noexcept;
```

This will copy up to 64 mask elements to an output value, storing each mask element in a single bit. Unfortunately this potentially loses bits, since `to_ullong` will only emit those bits which can be contained in a 64-bit representation. As with the unsigned_integral constructor this could be solved if the C23 bit-precise `_BitInt` type was available in C++.

§ 3.3. Implementation experience

In some problem domains, such as telecommunications, compact representations of bits as integers are very common and it is very important to be able to efficiently convert to and from this format. Providing an API to make it easy to convert masks to and from this representation proved invaluable in writing concise meaningful code in this problem domain.

On a machine with compact mask representation (e.g., we tested on AVX-512) the masks are already stored in compact form, so converting to and from an integer representation was trivial.

On a machine with wide mask representation (e.g., SSE, AVX, AVX2) it is not easy or efficient to use compact representation if only the standard `std::simd` API can be used (e.g., converting to a byte memory and then from that to individual bits). Efficient conversion is only possible if target-specific instructions are used, and the programmer writes non-portable code to use them. For these targets then, it was better that a uniform API into a compact format was provided and handled efficiently by `std::simd` itself.

Constructing and accessing bits through the bitset pathway (e.g., `mask.to_bitset().to_ullong()`) proved also to be inefficient under some conditions. Even on targets which already had compact mask representation, the extra step in storing data temporarily in a bitset added to the overhead of the conversion. The extra step was difficult to remove because data would be moved in/out of a bitmask in 64-bit blocks, and in/out of a simd_mask in blocks whose sizes were governed by the operation in progress. This mismatch in sizes made it difficult to smooth out the data flow across the conversion. Further work will be done to determine if better code generation is possible.

§ 4. Initialization list

Inevitably the programmer will want to be able to construct SIMD values with lists of known constants, such as for a lookup table. A reasonable way to achieve this would be to initializer-list syntax as follows:

```
simd<float> myLut = {2.f, 9.f, 23.f};
```

The first three elements values of the SIMD value will be initialized to the floating point values 2, 9 and 23 respectively. The remaining elements of the SIMD value will be value initialised.

To permit this syntax `std::simd` it is necessary to create an initializer-list constructor:

```
constexpr simd(std::initializer_list<value_type> list);
```

There is already a constructor which takes a single element and broadcasts it across the entire SIMD value. In combination with the initializer-list overload this means that the behaviour of SIMD initialisation from a list behaves as follows:

```
simd<int> a(1);       // [1, 1, 1, 1]
simd<int> b{1};       // [1, 1, 1, 1]
simd<int> c = {1};    // [1, 1, 1, 1]
simd<int> d{1, 0};    // [1, 0, 0, 0]
simd<int> e = {1, 0}; // [1, 0, 0, 0]
```

It is also convenient to be able to use type deduction on initialisation lists to construct a simd of the correct type without explicitly providing size or type arguments.

```
simd lut = {3.f, 5.f, 7.f};
// fixed_size_simd<float, 3>
```

§ 4.1. Design alternative - use variadic constructor

An alternative to an initialization list is to use a constructor which takes a variadic number of arguments and puts each argument into the respective SIMD value element position. Such a constructor would look like this:

```
template<typename... Us>
constexpr simd::simd(Us... us)
requires(sizeof...(Us) > 1);
```

This particular constructor has a constraint which differentiates it from the existing broadcast constructor. An alternative constraint could be to require exactly the right number of arguments as there are elements in the simd. Requiring an exact match in argument list length means that the user gets precisely the values in the simd that they asked for, without value initialisation of unspecified extra elements, but it doesn't allow a small number of values to be inserted into a native simd of unknown length unless the native size happens to be the same length:

```
simd<float> x = {1, 2, 3, 4}; // Compiles on Intel SSE, but not on Intel AV
```

This constructor is more complicated than the intialiser list since every argument would have to be constrained/converted individually, which introduces additional work. It also needs to be considered for overload resolution in every call to a constructor. The initializer list is simpler because it expects a list of uniform types.

§ 4.2. Implementation experience

The most common use case for the initialisation list is in creating compile-time lookup tables or data-parallel constants (e.g., FFT twiddles):

```
constexpr simd<float> twiddles = {pi_oneEighth, pi_threeEighth, pi_fiveEigh
```

The LUT will be created at compile time, so efficiency of generated code is not an issue here. Where the input is not constexpr, then the constructor mapped onto the efficient copy-from-memory constructor instead.

§ 5. Contiguous ranges

It is useful for `std::simd` to be able to read blocks of data stored in other types into a SIMD for data-parallel processing. To support this we propose that the following constructor is provided:

```
constexpr simd(std::ranges::contiguous_range auto x);
```

This can be used to construct a SIMD value from a variety of C++ types, such as `std::span`, `std::array`, and more.

As with the initialisation list, deduction guides can be provided too:

```
template <class T, std::size_t N> simd(std::span<T, N>) -> simd<T, N>;
template <class T, std::size_t N> simd(std::array<T, N>) -> simd<T, N>;
```

§ References

§ Informative References

[P1928R3]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 3 February 2023. URL: https://wg21.link/p1928r3

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: https://wg21.link/p2638r0